

01 — Codebase Architecture and Seams

What you'll learn: the relevant files, the exact integration points (with file:line), the data

flow, and the frame conventions you must respect.

Prereqs: 00-implementation-plan.

Why it matters here: so the GP code slots in correctly and safely.

1. The packages

- ``dvl_correction`` — Python (numpy-only), the navigation fusion library. All GP work lives here. `pip install -e ./dvl_correction; tests python3 -m unittest discover -s dvl_correction/tests`.

- ``imu-drift`` — Rust `no_std` crate, closed-form IMU drift mean/variance + Allan variance. Not wired into Python; relevant as the **edge GP prior** (references/05 §5).

- ``frontend`` — React/TS viewer + Rust WS stub server. Relevant only for the optional live demo (03-benchmark-and-frontend-demo).

2. Data flow (current)

```
raw sensors → NavigationFusionPipeline (synchronization.py)
               | buffers events in a timestamp-ordered heap (max_delay_s watermark)
               | interpolates IMU between samples
               ▼
DvlCorrectionLayer.correct (dvl_correction.py) — frame/scale/lever-arm + quality gates
               | → CorrectedDvlMeasurement(velocity_body_m_s, covariance_body, ...)
               ▼
(optional) DvlDeadReckoningTrack (dead_reckoning.py) — velocity → nav-position pseudo-measurement
               ▼
DvlImuKalmanLayer (dvl_imu_kalman.py) — error-state EKF: propagate (IMU) + update (DVL/mag)
               ▼
NavigationOutput (per step: position, velocity, attitude, biases, 16×16 covariance)
```

The GP inserts between "corrected DVL" and "EKF update," as a sibling to `DvlDeadReckoningTrack`.

3. The exact seams

Seam A — the measurement carries its own covariance (enables adaptive R_k)

`dvl_correction/src/dvl_imu_kalman.py:38-50:`

```
@dataclass(frozen=True)
class CorrectedDvlMeasurement:
    timestamp_s: float
    velocity_body_m_s: ArrayLike3 | None = None
    covariance_body: CovarianceLike3 | None = None      # ← set this to the GP predictive cov
    position_nav_m: ArrayLike3 | None = None
    position_covariance_nav: CovarianceLike3 | None = None
```

The velocity update consumes it (dvl_imu_kalman.py:213-230):

```
def update_corrected_dvl_velocity(self, measurement):
    velocity_body = _as_vec3(measurement.velocity_body_m_s, "velocity_body_m_s")
    measurement_covariance = _velocity_measurement_covariance(measurement, self.config) # ← uses covariance_body
    ...
    return self._apply_measurement_update(residual, h_matrix, measurement_covariance)
```

So **adaptive R_k** requires no EKF change — just populate covariance_body.

Seam B — outlier gating already exists

dvl_imu_kalman.py:277-300 (_apply_measurement_update) computes the innovation covariance and rejects updates whose **Mahalanobis distance** exceeds config.mahalanobis_gate (default 16.27). The GP's is_outlier is an additional, earlier pre-filter; this is the backstop.

Seam C — the pipeline injection points

dvl_correction/src/synchronization.py:

- _process_event handles raw_dvl (lines 136-149) and corrected_dvl (127-135). After correction

and before self.kalman.update_corrected_dvl(...), call the GP to **denoise + set covariance**.

- _add_dvl_track_position (lines 158-177) is the **template**: an optional component (self.dvl_track) that transforms/augments a CorrectedDvlMeasurement before it reaches the EKF.

The GP component mirrors this exactly.

- _propagate_filter_to (lines 179-185) advances the filter to a timestamp via interpolated IMU;

this is where, during DVL silence, the **bridge** logic injects a GP-predicted velocity update.

- Add a gp_velocity constructor arg next to dvl_track (lines 46-56), and counters to PipelineDiagnostics (lines 24-35): gp_denoised, gp_bridge_updates, gp_outlier_rejections.

Seam D — the dead-reckoning template (shape to mirror)

dvl_correction/src/dead_reckoning.py — DvlTrackConfig / DvlTrackState / DvlDeadReckoningTrack

with update(...) returning a state carrying position_nav_m + covariance_nav. The GP module copies

this dataclass-config + state + helper structure (and the numpy _as_vec3 / _as_covariance / _quat_to_rotation_matrix helpers, lines 91-122).

Seam E — config + exports

- dvl_correction/src/config.py build_navigation_pipeline has a _dvl_track block; add an analogous

gp_velocity block, **off by default**.

- dvl_correction/src/__init__.py exports the public symbols; add GpDvlVelocityModel,

GpVelocityConfig.

4. Frame & convention rules (do not violate)

- **DVL velocity is body-frame** (`velocity_body_m_s`); **position is nav-frame** (ENU). The GP models

velocity in **body frame** (same as the measurement) so its mean/cov drop straight into `update_corrected_dvl_velocity` with no rotation. (If you ever rotate a covariance, use $R \Sigma R^T$ as `dead_reckoning.py:70-74` does.)

- **Attitude is a wxyz quaternion** (`NavigationState.attitude_quat_wxyz`).

• **Timestamps are monotonic**; the pipeline rejects out-of-order events (`synchronization.py:103-108`). The GP must query/ingest in time order — it already receives events in order.

- **Covariances must be SPD**; symmetrize on output (the repo does $0.5*(C+C^T)$).

5. Tests to mirror

- `dvl_correction/tests/test_ekf_quality.py` — `_run_stationary_with_accel_bias` builds a synthetic

IMU+DVL scenario and asserts on drift; the GP benchmark reuses this pattern with dropouts.

- `dvl_correction/tests/test_dead_reckoning_and_cli.py` — pattern for testing a pipeline component.

Key takeaways

- Insert the GP **between corrected DVL and the EKF update**, mirroring `DvlDeadReckoningTrack`.
- **Adaptive** `Rk` = `set CorrectedDvlMeasurement.covariance_body`; **outlier gate** already exists.
- Wiring points: `synchronization.py _process_event / _add_dvl_track_position / _propagate_filter_to`; config in `config.py`; exports in `__init__.py`.
- Respect body/nav frames, wxyz quaternion, monotonic time, SPD covariances.

Further reading

- 02 — `gp_velocity` module design